Classification of Legal Text: Replication & Tuning

Mini-Project Write-up | Replicating Krithika Iyer, *"Classification of Legal Text,"* Stanford CS229 (2020)

Problem Statement

Legal opinions use long, non-standard sentence structures that break typical NLP tooling. We replicate and extend a Stanford CS229 project that classifies U.S. Supreme Court opinions into their human-curated legal subject-matter category, at two granularities: **issue_area** (15 broad categories, e.g. Criminal Procedure, Civil Rights, First Amendment) and **issue** (260+ fine-grained issue codes). The goal is to test whether classic topic-model and embedding-based document features, paired with a simple classifier, can recover these categories, and whether tuning their hyperparameters meaningfully improves on the paper's untuned baseline.

Dataset

We use the exact dataset the original paper used: the Supreme Court Database (Washington University) via the **textacy** package's bundled **SupremeCourt** dataset — 8,419 opinions (1946-2016), each with an expert-assigned issue_area and issue code. No substitute was needed. After dropping opinions with no issue_area label and issue codes with fewer than 10 examples (so every class can appear in train/dev/test), 7,971 opinions remain, split 70%/15%/15% into train/dev/test, stratified by issue_area. Opinions run long (median ~34,000 characters); each is truncated to its first 4,000 characters before vectorization, keeping LDA/Doc2Vec training tractable.

Approach

We replicate the paper's two completed experiments (its third, BERT, ran out of memory in the original work and produced no results to reproduce):

1. LDA + Logistic Regression — a gensim Dictionary and TF-IDF model feed a Latent Dirichlet Allocation topic model; each document becomes a dense topic-probability vector, classified with scikit-learn Logistic Regression.
2. Doc2Vec + Logistic Regression — a gensim Doc2Vec model learns a fixed-size paragraph embedding per opinion, again classified with Logistic Regression.

Phase 1 (replicate): both models trained with the paper's rough default settings (LDA num_topics=50, Doc2Vec vector_size=100, Logistic Regression C=1.0), evaluated on both label tasks. **Phase 2 (tune):** each feature extractor's main hyperparameter (LDA num_topics $\in$ {20, 50, 100}; Doc2Vec vector_size $\in$ {50, 100, 200}) is selected by issue_area dev accuracy, then each downstream classifier's regularization C is grid-searched per label task on dev only — the test set is touched exactly once, with the final chosen configuration.

Implementation

Code: `src/preprocessing.py` (loading/cleaning/splitting), `src/lda_lr.py` / `src/doc2vec_lr.py` (feature extractors), `src/classifiers.py` (SVM/Random Forest/Naive Bayes), `run_all.py` / `tune_all.py` / `compare_classifiers.py` (Phases 1/2/3). All hyperparameter and classifier selection uses only the dev split; test is evaluated once per final configuration.

Results — Phase 1 (paper's default settings)

Model	Label task	Test Acc	Test Macro-F1
lda_lr	issue_area (15-label)	39.3%	0.149
lda_lr	issue (fine-grained)	8.3%	0.013
doc2vec_lr	issue_area (15-label)	59.1%	0.463
doc2vec_lr	issue (fine-grained)	37.1%	0.290

Results — Phase 2 (tuned, test touched once)

Model + Task	Tuned Param	Test Acc Before	Test Acc After

lda_lr_issue_area	num_topics=100	39.3%	48.7%
lda_lr_issue	num_topics=100	8.3%	19.9%
doc2vec_lr_issue_area	vector_size=50	59.1%	62.2%
doc2vec_lr_issue	vector_size=50	37.1%	42.0%

LDA benefited from more topics (50→100; issue_area test accuracy +9.4pp) — 50 was too coarse for 15+ categories. Doc2Vec preferred a *smaller* embedding (100→50): with ~5,600 training documents, 200 dimensions overfits before the classifier sees enough signal.

Results — Phase 3: classifier comparison (beyond the paper)

The paper only tries Logistic Regression. We additionally compare **SVM** (RBF kernel), **Random Forest**, and **Naive Bayes (Gaussian)** on the identical Phase-2-tuned feature vectors, isolating classifier choice from feature extraction, on the issue_area task:

Classifier	Feature	Test Acc	Test Macro-F1
svm	doc2vec	68.8%	0.499
logistic_regression	doc2vec	62.2%	0.477
random_forest	doc2vec	59.6%	0.317
naive_bayes	doc2vec	58.7%	0.462
svm	lda	49.1%	0.266
logistic_regression	lda	48.7%	0.267
random_forest	lda	46.1%	0.253
naive_bayes	lda	13.1%	0.137

SVM is the overall best model: 68.8% test accuracy on Doc2Vec+issue_area, beating Logistic Regression's 62.2% on the same features (+6.6pp) — the RBF kernel draws a non-linear boundary LR's linear one can't. Random Forest and Naive Bayes both underperform LR on Doc2Vec; Naive Bayes also collapses on LDA features (13% accuracy), since its per-feature Gaussian assumption fits topic-probability vectors poorly.

Conclusions

Doc2Vec embeddings consistently outperform LDA topic-distribution features, and the coarser 15-category task is much easier than the 260+-category fine-grained task — unsurprising given how thin per-class training data gets at that granularity. LDA's macro-F1 trails its accuracy badly (0.267 vs 48.7% on issue_area), defaulting to majority classes (Criminal Procedure, Economic Activity) and failing on rare ones; Doc2Vec generalizes far better to minority categories such as Privacy and Attorneys. Classifier choice mattered almost as much as feature extraction: SVM beat every other classifier tried. Our replicated numbers land in the same range and order as the original paper's. We note an internal inconsistency in the paper's own results table (its printed table and prose disagree on which label granularity scored higher) and resolve it in favor of the prose. We did not replicate BERT: the paper's own attempt failed with an out-of-memory error and left no baseline to reproduce; a long-document Transformer (e.g. Longformer) is natural future work, as the paper itself suggests.

Full code, results, and figures: github.com/amogh-herle/ML-MINI-PROJECT